```
## Pleae ensure you are using GPU for faster computataion, perferably run on google c
import torch
print("GPU available:", torch.cuda.is_available())
```

```
GPU available: True
```

```
!pip install sentence-transformers
!pip install scikit-learn
!pip install pandas
!pip install spacy
!pip install python-Levenshtein
```

```
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-Levenshtein in /usr/local/lib/python3.12/dist-
Requirement already satisfied: Levenshtein==0.27.3 in /usr/local/lib/python3.12/dist
Requirement already satisfied: rapidfuzz<4.0.0,>=3.9.0 in /usr/local/lib/python3.12/
```

To grade Spanish learner responses accurately,Roberata based **es_dep_news_tr** is used here .Though computationally intensive, it gives high accuracy for scoring meaning over grammar.

```
!python -m spacy download es_dep_news_trf
```

```
Collecting es-dep-news-trf==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/es_dep_news_
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 407.8/407.8 MB 4.5 MB/s eta 0:00:00
Requirement already satisfied: spacy-curated-transformers<1.0.0,>=0.2.2 in /usr/local/
Requirement already satisfied: curated-transformers<0.2.0,>=0.1.0 in /usr/local/lib/py
Requirement already satisfied: curated-tokenizers<0.1.0,>=0.0.9 in /usr/local/lib/pyth
Requirement already satisfied: torch>=1.12.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: regex>=2022 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (fror
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/pytho
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/pyt
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/pytho
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/pytho
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/pytho
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/d
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/pytho
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packa
✓ Download and installation successful
You can now load the package via spacy.load('es_dep_news_trf')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Here, I am evaluating **how much key information from a target sentence appears in a participant's response**. In language assessment tasks like eit, it is important to measure whether the response **preserves the main meaning or idea units** of the target.

To achieve this, the code performs three main steps:

1. Text Normalization

Transcriptions of spoken responses often contain fillers, annotations, pauses, and inconsistent punctuation.
The preprocessing step cleans the text by:

- Removing transcription markers and fillers
- Standardizing casing
- Removing punctuation and extra spaces

This ensures that comparisons are based only on **meaningful linguistic content**.

2. Extraction of Content Words

Instead of comparing every word, the method focuses on **content-bearing words** (such as nouns, verbs, adjectives, and adverbs).
These words typically represent the **core semantic information** of a sentence, making them suitable proxies for **idea units**.

3. Information Coverage Measurement

Coverage =

$$\frac{\text{Shared content words}}{\text{Content words in the target}}$$

A higher score indicates that the response preserves more of the key information from the target.

```
import spacy
import Levenshtein
import pandas as pd
import re
import string
nlp = spacy.load("es_dep_news_trf")

def content_words(sentence):
    doc = nlp(sentence)
    return [token.lemma_ for token in doc if token.pos_ in ["NOUN", "VERB", "PROPN",

def coverage(target, response):
    """
    Calculates the percentage of target Idea Units found in the response.
    """
    t=content_words(target)
    r=content_words(response)
```

```
    if not t:
        return 0.0
    overlap = set(t).intersection(set(r))
    coverage_score = len(overlap) / len(t)
    return coverage_score

def clean_transcription(text):
    if pd.isna(text): return ""
    text = str(text).lower()
    text = re.sub(r'\[.*?\]', '', text)
    text = re.sub(r'\b(xxx|um|uh|eh|ahm|mhh)\b', '', text)
    text = re.sub(r'\(\d+\)', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation + '¿?¡!'))
    return ' '.join(text.split()).strip()

def edit_distance_ratio(s1,s2):
    return Levenshtein.ratio(s1,s2)
```

Computes **semantic similarity** between two sentences using sentence embeddings.
The similarity score (cosine similarity) indicates how close the meanings of the two sentences are.

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

def semantic_similarity(s1, s2):
    emb1 = model.encode(s1)
    emb2 = model.encode(s2)
    return cosine_similarity([emb1],[emb2])[0][0]
```

```
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 199/199 [00:00<00:00, 643.50it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok
```

```
## DEMO1: the model idetifies the key information in each sentence
test_target = "Las calles de esta ciudad son muy anchas"
test_response = "Las calles son muy grandes"

print("Target Idea Units:", content_words(test_target))
# Expected: ['calle', 'ciudad', 'mucho', 'ancho']

print("Response Idea Units:", content_words(test_response))
# Expected: ['calle', 'mucho', 'grande']
```

```
Target Idea Units: ['calle', 'ciudad', 'mucho', 'ancho']
Response Idea Units: ['calle', 'mucho', 'grande']
```

```
## DEMO 2: Semantic Equivalence (Synonyms)
# "Cars" vs "Vehicles" and "Red" vs "Crimson"
test_target = "Los coches son rojos"
test_response = "Los vehículos son de color carmesí"

# Show that while Idea Units might differ, the Semantic Score is high
print("Target Idea Units:", content_words(test_target))
print("Response Idea Units:", content_words(test_response))
print("Semantic Similarity Score:", semantic_similarity(test_target, test_response))
# Expected: High score (>0.80) despite different lemmas
```

```
Target Idea Units: ['coche', 'rojo']
Response Idea Units: ['vehículo', 'color', 'carmesí']
Semantic Similarity Score: 0.57192945
```

```
## DEMO 3: This shows how the model recognizes the "Idea" even if the grammar changes.
test_target = "El niño camina hacia la escuela" # Lemma: caminar
test_response = "El niño caminó a la escuela"   # Lemma: caminar

print("Target Lemmas:", content_words(test_target))
print("Response Lemmas:", content_words(test_response))

# This will be 1.0 because 'camina' and 'caminó' both share the lemma 'caminar'
print("Idea Unit Match:", coverage(test_target, test_response))
```

```
Target Lemmas: ['niño', 'caminar', 'escuela']
Response Lemmas: ['niño', 'caminar', 'escuela']
Idea Unit Match: 1.0
```

Sentence Scoring Function

This function assigns a **score from 0 to 4** to a participant's response based on how closely it matches the target sentence.

First, both the **target** and **response** are cleaned. The function then computes three measures: **semantic similarity**, **content coverage**, and **edit distance similarity**.

The final score is assigned using the following criteria:

- **4 – Exact Match:** The response is almost identical to the target (very high edit similarity).
- **3 – Meaning Preserved:** The response conveys the same meaning with different wording (high semantic similarity and high idea coverage).
- **2 – Close Meaning:** The response preserves more than half of the key information.
- **1 – Partial/Unrelated:** Only a small portion of the target meaning is present.
- **0 – Abandoned/Silence:** Very little meaningful content or no response.

```
def score_sentence(target_raw, response_raw):
    target = clean_transcription(target_raw)
    response = clean_transcription(response_raw)

    sim = semantic_similarity(target, response)
    cov = coverage(target, response)
    ed  = edit_distance_ratio(target, response)

    # SCORE 4: Exact Match (Only rely on Edit Distance to allow for minor spelling ty
    if ed >= 0.95:
        return 4
    # SCORE 3: Meaning Preserved (High similarity and high coverage of idea units, bu
    elif sim > 0.75 and cov > 0.70:
        return 3
    # SCORE 2: Close Meaning (More than half of idea units preserved)
    elif sim > 0.50 and cov >= 0.45:
        return 2
    # SCORE 1: Partial/Unrelated (Less than half of idea units preserved)
    elif cov >= 0.20 or sim > 0.25:
        return 1
    # SCORE 0: Abandoned/Silence (Minimal words, garbled)
    else:
        return 0
```

Automatic Scoring of EIT Responses

This section applies the scoring pipeline to an Excel file containing **Elicited Imitation Task (EIT)** responses. The file may contain multiple sheets, each representing a different participant.

For each sheet:

1. The script reads the **stimulus sentence** and the **participant's transcription**.
2. Each pair is passed through the scoring pipeline, which:
    - Cleans the text
    - Computes similarity and coverage measures
    - Assigns a score from **0–4** using the `score_sentence` function.
3. The computed score is added to the **Score** column.

Finally, all scored sheets are saved

```
from google.colab import files
import io
print("Please upload the Excel file from your local machine.")
uploaded = files.upload()
input_file_path = None
for filename in uploaded.keys():
    print(f'User uploaded file "{filename}"')
    with open(filename, 'wb') as f:
        f.write(uploaded[filename])
    input_file_path = filename
    break
```

```
if input_file_path is None:
    raise FileNotFoundError("No file was uploaded. Please try again.")
output_file = "AutoEIT_Final_Scored_Output.xlsx"
all_sheets = pd.read_excel(input_file_path, sheet_name=None)

with pd.ExcelWriter(output_file, engine='openpyxl') as writer:
    for sheet_name, df in all_sheets.items():
        print(f"Scoring participant: {sheet_name}...")
        if 'Stimulus' in df.columns and 'Transcription Rater 1' in df.columns:
            df['Score'] = df.apply(
                lambda x: score_sentence(x['Stimulus'], x['Transcription Rater 1']),
                axis=1
            )
            df.to_excel(writer, sheet_name=sheet_name, index=False)
        else:
            print(f"Skipping sheet '{sheet_name}' as 'Stimulus' or 'Transcription Rate

print(f"\nSuccess! Final multi-sheet submission file saved as: {output_file}")
```

```
Please upload the Excel file from your local machine.
```

Choose Files AutoEIT Sa…Scoring.xlsx

AutoEIT Sample Transcriptions for Scoring.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 60255 bytes, last modified: 3/6/2026 - 100% done

```
Saving AutoEIT Sample Transcriptions for Scoring.xlsx to AutoEIT Sample Transcriptions
User uploaded file "AutoEIT Sample Transcriptions for Scoring (2).xlsx"
Scoring participant: Info...
Skipping sheet 'Info' as 'Stimulus' or 'Transcription Rater 1' columns are missing.
Scoring participant: 38001-1A...
Scoring participant: 38002-2A...
Scoring participant: 38004-2A...
Scoring participant: 38006-2A...

Success! Final multi-sheet submission file saved as: AutoEIT_Final_Scored_Output.xlsx
```